

Les fonctions en JavaScript

Les fonctions sont des blocs de code réutilisables qui effectuent une tâche spécifique. Elles permettent de structurer le code, de le rendre plus lisible et de faciliter la maintenance.

Déclaration d'une fonction

Une fonction est déclarée à l'aide du mot-clé `function`, suivi du nom de la fonction, de parenthèses (qui peuvent contenir des paramètres) et d'accolades qui délimitent le corps de la fonction.

ATTENTION: La norme veut que le nom de la fonction commence par un verbe d'action et respecte les mêmes règles de nommage que les variables (pas d'espaces, pas de caractères spéciaux, ne pas commencer par un chiffre, etc.).

```
function maFonction(param1, param2) {  
    // Corps de la fonction  
    console.log("Paramètre 1:", param1);  
    console.log("Paramètre 2:", param2);  
}
```

Appel d'une fonction

Pour déclencher l'exécution d'une fonction, il suffit de l'appeler en utilisant son nom suivi de parenthèses, en passant les arguments nécessaires.

```
maFonction("Bonjour", "Monde");
```

Dans cet exemple, la fonction `maFonction` est appelée avec les arguments "Bonjour" et "Monde", ce qui affichera ces valeurs dans la console.

Valeur de retour d'une fonction

Certaines fonctions peuvent effectuer des opérations et retourner une valeur à l'endroit où elles ont été appelées. Pour cela, on utilise le mot-clé `return`. On placera la fonction dans une variable pour récupérer la valeur retournée.

Les valeurs de retour peuvent être de n'importe quel type : nombre, chaîne de caractères, objet, tableau, etc.

IMPORTANT: La fonction s'arrête dès qu'elle rencontre un `return`. Tout code après un `return` dans une fonction ne sera pas exécuté.

```
function additionner(a, b) {  
    return a + b;  
}
```

```
let resultat = additionner(5, 3);
console.log("Résultat de l'addition:", resultat); // Affiche 8
```

Exemple: états de jeu

Il est possible de regrouper ce qui doit être fait dans différents états de jeu en utilisant des fonctions. Par exemple, on peut avoir une fonction pour l'état "menu", une autre pour l'état "jeu" et une autre pour l'état "pause". On appellera la fonction appropriée en fonction de l'état actuel du jeu avec les parenthèses.

```
let etatJeu = "menu"; // Peut être "menu", "jeu", ou "pause"

function initialiserJeu() {
    setInterval(boucleDeJeu, 1000 / 60); // Démarre la boucle de jeu
}

function boucleDeJeu() {
    oContexte.clearRect(0, 0, oCanvasHTML.width, oCanvasHTML.height); // Efface le canvas
    if (etatJeu === "menu") {
        afficherMenu();
    } else if (etatJeu === "jeu") {
        demarrerJeu();
    } else if (etatJeu === "pause") {
        afficherPause();
    }
}

function afficherMenu() {
    // Code pour afficher le menu
    console.log("Affichage du menu");
}

function demarrerJeu() {
    // Code pour démarrer le jeu
    console.log("Démarrage du jeu");
}

function afficherPause() {
    // Code pour afficher l'écran de pause
    console.log("Jeu en pause");
}
```

Exemple: Fonction de collision

Voici un exemple simple d'une fonction qui vérifie si un curseur est en collision avec un élément sur le canvas. Lorsqu'une collision est détectée, la fonction retourne **true**, sinon elle retourne **false**. On enregistrera le résultat dans une variable pour l'utiliser plus tard.

```

let oCanvasHTML = document.querySelector("monCanvas");
let oContexte = oCanvasHTML.getContext("2d");

let positionXBouton = 50;
let positionYBouton = 50;
let largeurBouton = 100;
let hauteurBouton = 50;

function initialiser() {
    oCanvasHTML.addEventListener("click", clicCanvas);
}

function clicCanvas(evenement) {
    let nCurseurX = evenement.clientX;
    let nCurseurY = evenement.clientY;

    // ICI on appelle la fonction et on stocke le résultat dans une variable
    let bCollision = estEnCollision(
        nCurseurX,
        nCurseurY,
        positionXBouton,
        positionYBouton,
        largeurBouton,
        hauteurBouton
    );

    if (bCollision) {
        console.log("Le bouton a été cliqué !");
    } else {
        console.log("Pas de collision avec le bouton.");
    }
}

function estEnCollision(curseurX, curseurY, elementX, elementY, elementLargeur,
elementHauteur) {
    // Si la position du curseur est à droite de l'élément
    // ET si la position du curseur est aussi à gauche de l'élément
    // ET si la position du curseur est aussi en dessous de l'élément
    // ET si la position du curseur est aussi au dessus de l'élément
    // ALORS il y a collision
    let bEstAdroite = curseurX >= elementX;
    let bEstAGauche = curseurX <= elementX + elementLargeur;
    let bEstEnHaut = curseurY >= elementY;
    let bEstEnBas = curseurY <= elementY + elementHauteur;

    let bEstEnCollision = bEstAdroite == true && bEstAGauche == true && bEstEnHaut
== true && bEstEnBas == true;

    console.log("Collision détectée:", bEstEnCollision);

    // On retourne le résultat de la collision, la fonction s'arrête ici
    return bEstEnCollision;
}

```

